

It only makes sense to use the frame clause if the order by clause is also present. We will use the ROWS BETWEEN clause when we are going to consider a specific set of records relative to the current row. We will use the RANGE BETWEEN clause when we are going to consider a range of values in a specific column relative to the value in the current row.

ROWS BETWEEN start_point and end_point

Now we will look at some simple examples to try to better explain the frame_set clauses. These are typically used to do in-depth data analysis and data mining, among other tasks. Let's start with some examples, beginning here:

```
forumbd=> select x from (select generate_series(1,5) as x) V WINDOW w as
(order by x) ;
x
---
1
2
3
4
5
(5 rows)
```

Suppose that we want to have an incremental sum row by row. The goal that we want to reach is as follows:

x	sum(x)
1	1
2	3
3	6
4	10
5	15

This can be achieved using the following query:

```
forumbd=> select x, sum(x) over (order by x) from generate_series(1,5) as
x;
x | sum
---+-----
1 | 1
2 | 3
3 | 6
```

```

4 | 10
5 | 15
(5 rows)

```

The same query can be written in this way:

```

forumbd=> SELECT x, SUM(x) OVER w
FROM (select generate_series(1,5) as x) v
WINDOW w AS (ORDER BY x ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW);
x | sum
---+-----
1 | 1
2 | 3
3 | 6
4 | 10
5 | 15
(5 rows)

```

Now, let's imagine that the query was executed in successive steps, one for each row of the table. In the following diagrams, we will simulate the internal behavior of PostgreSQL, to better understand how the clause `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` works:

1. First, PostgreSQL uses the `order_by_clause` condition to order the data inside the window, as shown by the blue arrow in the following diagram:

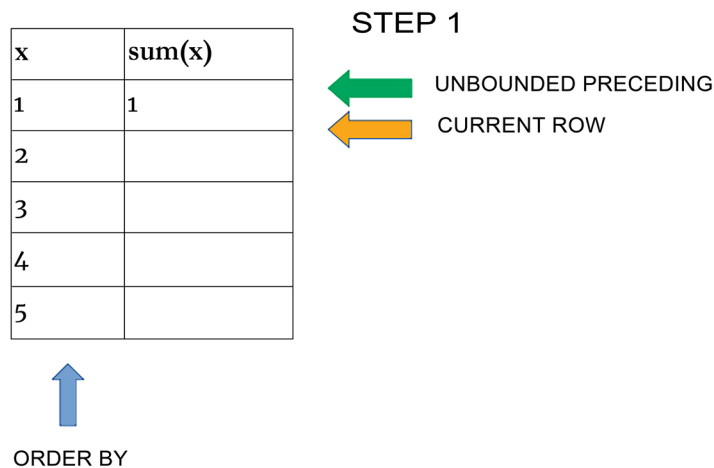


Figure 6.3: The order by clause

As we can see, on the right of the image, we have two further pointers: a green one for the **UNBOUNDED PRECEDING** clause and an orange pointer for the **CURRENT ROW** clause. The result is 1, so in the first step both point to the first row. Now, let's see what happens in the next steps.

- 2. In this step, the **UNBOUNDED PRECEDING** pointer still points to the first row, whereas the **CURRENT ROW** pointer now points to the second row, and the result of the sum is $1+2 = 3$:

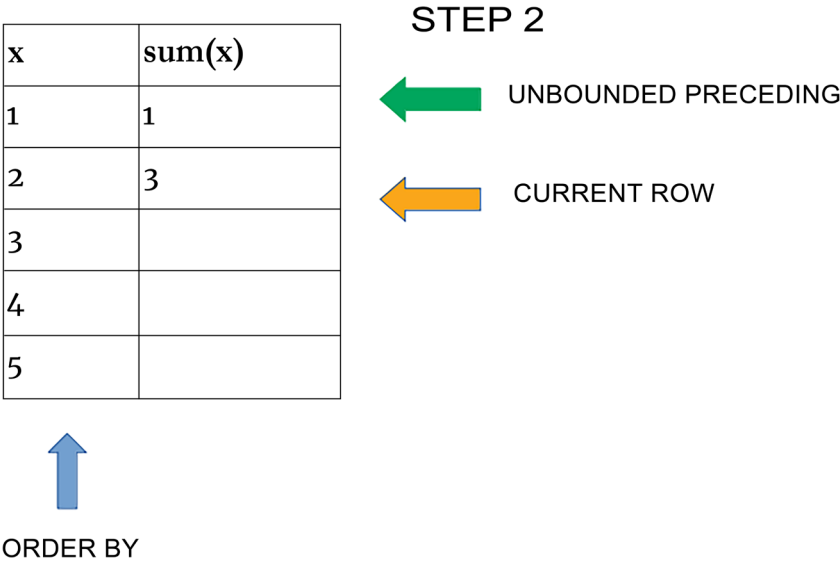


Figure 6.4: The unbounded preceding and current row (1)

- 3. Next, the **UNBOUNDED PRECEDING** pointer still points to the first row, whereas the **CURRENT ROW** pointer points to the third row, and the result of the sum is $1+2+3 = 6$:

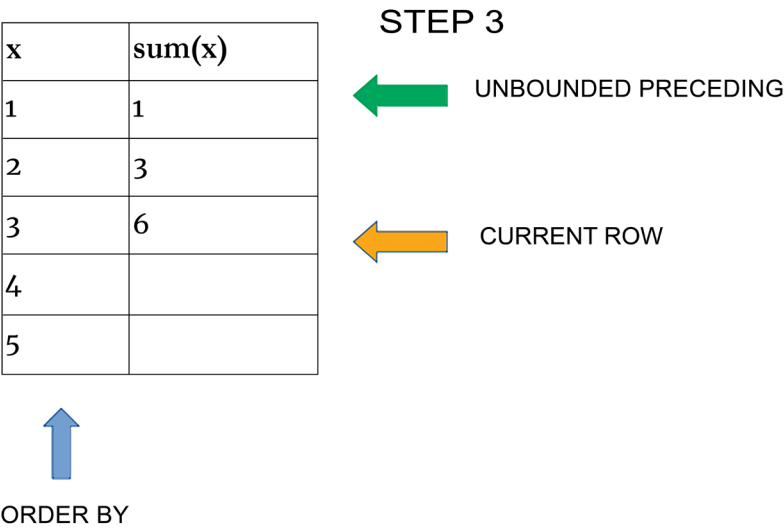


Figure 6.5: The unbounded preceding and current row (2)

4. In the fourth step, the **UNBOUNDED PRECEDING** pointer still points to the first row, whereas the **CURRENT ROW** pointer now points to the fourth row, and the result of the sum is $1+2+3+4 = 10$:

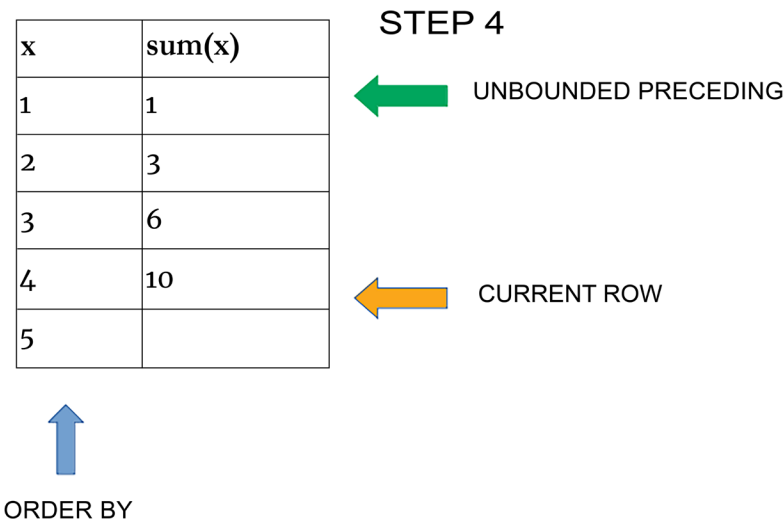


Figure 6.6: The unbounded preceding and current row (3)

5. And in the fifth and final step, we have the desired result:

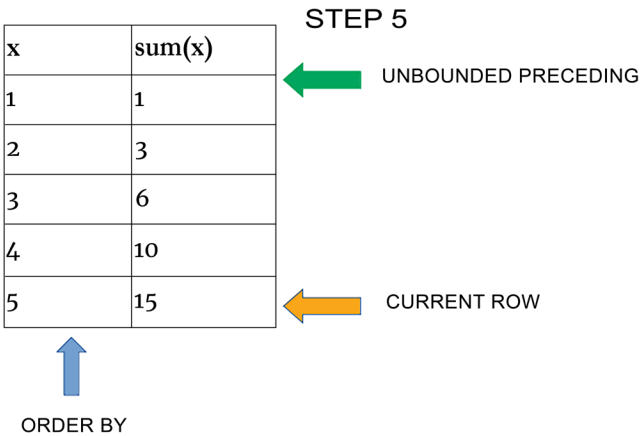


Figure 6.7: The unbounded preceding and current row (4)

That is how a frameset clause works!

Let’s look at some more examples of how the frame clause works using different options. If for each row of the table we wanted to find the sum of the current row with the preceding row, we would start with the following:

X
1
2
3
4
5

We want to end up with the following result:

x	sum(x)
1	1
2	3
3	5
4	7
5	9

The query that we have to perform is described in the following example:

```
forumbd=> SELECT x, SUM(x) OVER w
FROM (select generate_series(1,5) as x) V
WINDOW w AS (ORDER BY x RANGE BETWEEN 1 PRECEDING AND CURRENT ROW);
```

x	sum
1	1
2	3
3	5
4	7
5	9

(5 rows)

The preceding query works similarly to what we saw before. The only difference is that now the calculation range is between the first row and the current row of the partition, as written in the statement `BETWEEN 1 PRECEDING AND CURRENT ROW`. In this example, only two lines are used to calculate the sum. The same mechanism can be used to perform an incremental sum, as we can see in the following example:

```
forumbd=> SELECT x, SUM(x) OVER w
FROM (select generate_series(1,5) as x) V
WINDOW w AS (ORDER by x ROWS UNBOUNDED PRECEDING);
```

x	sum
1	1
2	3
3	6
4	10
5	15

(5 rows)

Now the only difference is that the calculation range is by `ROWS UNBOUNDED PRECEDING` and not `BETWEEN 1 PRECEDING AND CURRENT ROW`.

Let's look at another example where window functions simplify our work. Always starting from the series that we've seen before, we know that the total sum is $1+2+3+4+5 = 15$. So now suppose that we want to do a reverse sum starting from the max value of the table, that is, 5.

In this example, we want the result to be as follows:

x	sum(x)
1	15
2	14
3	12
4	9
5	5

The query that makes this possible is the following:

```
forumbd=> SELECT x, SUM(x) OVER w
FROM (select generate_series(1,5) as x) V
WINDOW w AS (ORDER BY x ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING);
 x | sum
---+---
 1 | 15
 2 | 14
 3 | 12
 4 |  9
 5 |  5
(5 rows)
```

What makes this possible is the **UNBOUNDED FOLLOWING** clause, which works the opposite way to **UNBOUNDED PRECEDING**. This happens because of the following:

- In the first row, all values are added: $1+2+3+4+5 = 15$.
- In the second row, these values are added: $2+3+4+5 = 14$.
- In the third row, these values are added: $3+4+5 = 12$.

RANGE BETWEEN start_point and end_point

As discussed earlier, when we use **RANGE BETWEEN**, we will consider a **RANGE** of values with respect to the value in the current row. The difference when it comes to the **ROWS** clause is that if the field that we use for **ORDER BY** does not contain unique values for each row, then **RANGE** will combine all the rows it comes across with non-unique values, rather than processing them one at a time.

In contrast, ROWS will include all of the rows in the non-unique bunch but process each of them separately:

1. First of all, let's create a simple dataset with duplicate data:

```
forumbd=> select generate_series(1,8) % 4 as x order by 1;
x
---
0
0
1
1
2
2
3
3
(8 rows)
```

2. Now let's do some tests to observe the differences between the ROWS and RANGE clauses. Let's start with the ROWS clause:

```
forumbd=> SELECT x, row_number() OVER w, SUM(x) OVER w FROM (select
generate_series(1,8) % 4 as x) V
WINDOW w AS (ORDER BY x ROWS BETWEEN 1 PRECEDING AND CURRENT ROW);
x | row_number | sum
---+-----+-----
0 |          1 |    0
0 |          2 |    0
1 |          3 |    1
1 |          4 |    2
2 |          5 |    3
2 |          6 |    4
3 |          7 |    5
3 |          8 |    6
(8 rows)
```

The preceding query works exactly as we've seen before; it sums the previous row with the current row.

3. Let's now see what happens if we use the RANGE clause instead of the ROWS clause:

```

forumbd=> SELECT x, row_number() OVER w, SUM(x) OVER w
FROM (select generate_series(1,8) % 4 as x) V
WINDOW w AS (ORDER BY x RANGE BETWEEN 1 PRECEDING AND CURRENT ROW);
 x | row_number | sum
---+-----+---
 0 |          1 |  0
 0 |          2 |  0
 1 |          3 |  2
 1 |          4 |  2
 2 |          5 |  6
 2 |          6 |  6
 3 |          7 | 10
 3 |          8 | 10
(8 rows)

```

Let's take this result:

x	row_number	sum
0	1	0
0	2	0
1	3	2
1	4	2
2	5	6
2	6	6
3	7	10
3	8	10

Now let's look at the result from the frame point of view:

x	row_number	sum	Frame Number
0	1	0	1
0	2	0	1
1	3	2	2
1	4	2	2
2	5	6	3

2	6	6	3
3	7	10	4
3	8	10	4

As we can see, there are four frames in the last table, so internally, PostgreSQL works in this way: first, PostgreSQL splits the window function into frames using the `order by` clause and then aggregates the data among the frames; for example:

- The sum of row number 3 is the result of the sum of row number 1 + row number 2 + row number 3 + row number 4: $0+0+1+1=2$.
- The sum of row number 4 is the result of the sum of row number 1 + row number 2 + row number 3 + row number 4: $0+0+1+1=2$.
- The sum of row number 5 is the result of the sum of row number 3 + row number 4 + row number 5 + row number 6: $1+1+2+2=6$.
- The sum of row number 6 is the result of the sum of row number 3 + row number 4 + row number 5 + row number 6: $1+1+2+2=6$.

In the preceding example, we have considered a partition ordered in an ascending way. In the next example, the partition is sorted in a descending way and we will see the difference between `ROWS` and `RANGE` in this scenario.

This is the query for the `RANGE` clause:

```
forumdb=> SELECT x,row_number() OVER w, dense_rank() OVER w,sum(x) OVER w
FROM (select generate_series(1,8) % 4 as x) V
WINDOW w AS (ORDER BY x desc RANGE BETWEEN 1 PRECEDING AND CURRENT ROW);
 x | row_number | dense_rank | sum
---+-----+-----+---
 3 |          1 |          1 |   6
 3 |          2 |          1 |   6
 2 |          3 |          2 |  10
 2 |          4 |          2 |  10
 1 |          5 |          3 |   6
 1 |          6 |          3 |   6
 0 |          7 |          4 |   2
 0 |          8 |          4 |   2
(8 rows)
```